



1^η Προγραμματιστική Εργασία

Τεχνητή Νοημοσύνη 2025

Διδάσκων:

Ι. Ανδρουτσόπουλος

Μέλη ομάδας:

Ευαγγελία Αγγέλου – p3230001

Αναστασία Κοκκάλα – p3230077

Φωτεινή Σφακιανάκη – p3230202

1. Τρόπος χρήσης

Το παιχνίδι Reversi έχει γραφτεί σε java και εκτελείται με τη βοήθεια της κλάσης Main, η οποία περιέχει την κύρια συνάρτηση main που περιγράφει τη διαδικασία του παιχνιδιού, κατά τη διάρκεια μιας παρτίδας. Στην αρχή του παιχνιδιού ζητείται από τον παίκτη να επιλέξει:

a. Τη σειρά που θα παίξει:

- Εισάγει τον αριθμό 1, αν επιθυμεί να παίξει πρώτος και παίρνει τα μαύρα πιόνια.
- Εισάγει τον αριθμό 0, αν επιθυμεί να παραχωρήσει τη σειρά του στον αντίπαλο (δηλαδή τον υπολογιστή) και να παίξει δεύτερος παίρνοντας έτσι τα άσπρα πιόνια

b. Το μέγιστο βάθος αναζήτησης

- Ο χρήστης εισάγει έναν ακέραιο αριθμό, με βάση τον οποίον εργάζεται ο αλγόριθμος Minimax. Ειδικότερα, όσο μεγαλύτερος είναι ο αριθμός τόσο καλύτερη θα είναι η επιλογή για την κίνηση των πιονιών αλλά και πιο χρονοβόρα.

Αφού κάνει αυτές τις δύο επιλογές ο χρήστης, τότε εμφανίζεται η γραφική διεπαφή του χρήστη, η οποία απεικονίζει το ταμπλό του παιχνιδιού και υλοποιείται από την κλάση GUI.java. Το αρχικό ταμπλό περιέχει τα 4 πρώτα βασικά πιόνια του παιχνιδιού. Σε περίπτωση που είναι η σειρά του παίκτη «Ανθρώπου» τότε οι έγκυρες δυνατές κινήσεις του εμφανίζονται με μία κουκίδα σε κάθε κελί. Από αυτές ο παίκτης κλικάρει στο κελί που επιθυμεί να τοποθετήσει το πιόνι του. Αντίθετα, όταν πρόκειται να παίξει ο AI παίκτης, δηλαδή να απαντήσει ο υπολογιστής, τότε εφαρμόζεται ο αλγόριθμος Τεχνητής Νοημοσύνης Minimax. Παράλληλα, οι κινήσεις των παικτών εμφανίζονται και στο τερματικό. Τέλος, όταν γεμίσει το ταμπλό ή δεν υπάρχουν πια διαθέσιμες έγκυρες κινήσεις, τότε το παιχνίδι τελειώνει και εμφανίζεται το τελικό σκορ και ο νικητής του παιχνιδιού.

2. Δυνατότητες

a. Επιλογή παραμέτρων παιχνιδιού:

Ο χρήστης μπορεί κατά την έναρξη της εφαρμογής να επιλέξει:

- Αν θα παίξει πρώτος ή δεύτερος.
- Το μέγιστο βάθος αναζήτησης του αλγορίθμου MiniMax, επιτρέποντας ρύθμιση της υπολογιστικής προσπάθειας του αντιπάλου.

b. Υλοποίηση αλγορίθμου MiniMax με άλφα-βήτα πριόνισμα

- Η επιλογή της κίνησης του υπολογιστή υλοποιείται εξ' ολοκλήρου από δική μας υλοποίηση του αλγορίθμου MiniMax.
- Εφαρμόζεται βελτιστοποίηση με πριόνισμα α - β , ώστε να μειώνεται ο αριθμός των καταστάσεων που εξετάζονται και να αυξάνεται η ταχύτητα απόφασης.
- Ο υπολογιστής αξιολογεί όλες τις επιτρεπτές κινήσεις σε βάθος, χρησιμοποιώντας κατάλληλη ευρετική συνάρτηση για την εκτίμηση πλεονεκτήματος.

c. Έλεγχος εγκυρότητας κινήσεων

- Το πρόγραμμα ελέγχει αυτόματα αν μια κίνηση του χρήστη είναι σύμφωνη με τους κανόνες του Reversi.
- Κάθε παράνομη κίνηση απορρίπτεται και εμφανίζεται κατάλληλο μήνυμα.

d. Αυτόματη διαχείριση περιπτώσεων χωρίς διαθέσιμες κινήσεις

- Αν ο παίκτης που έχει σειρά δεν μπορεί να τοποθετήσει πουθενά πούλι με νόμιμο τρόπο εμφανίζεται ενημερωτικό μήνυμα, και η σειρά περνάει αυτόματα στον αντίπαλο ή αν είναι η τελευταία κίνηση του παιχνιδιού τερματίζει το παιχνίδι.

e. Αναπαράσταση και ενημέρωση της κατάστασης του παιχνιδιού

- Η κατάσταση του παιχνιδιού εμφανίζεται μετά από κάθε κίνηση.
- Χρησιμοποιείται γραφική διεπαφή προσιτή προς τον χρήστη και εύκολη στην διαχείριση.
- Σωστή αλλαγή και αναστροφή δίσκων
- Μετά από κάθε έγκυρη κίνηση, το πρόγραμμα αναστρέφει όλα τα αντίπαλα πούλια που περικλείονται, σύμφωνα με τους κανόνες.

f. Αυτόματος τερματισμός παρτίδας και εμφάνιση αποτελέσματος

- Το παιχνίδι τερματίζει αυτόματα όταν δεν υπάρχουν διαθέσιμες κινήσεις για κανέναν παίκτη ή όταν ο πίνακας είναι πλήρης.
- Το πρόγραμμα υπολογίζει και εμφανίζει το τελικό σκορ και τον νικητή.

3. Αρχιτεκτονική

Το πρόγραμμα είναι χωρισμένο σε 6 κλάσεις:

Κλάση Move

Αρχικά υλοποιήσαμε την κλάση Move, η οποία περιγράφει μία κίνηση στο ταμπλό. Κάθε παιδί ενός κόμβου αντιπροσωπεύει μία κίνηση. Εκείνη αποθηκεύει τη γραμμή (**row**), τη στήλη (**col**) και την αξία (**value**) της κίνησης. Κατασκευάσαμε τον constructor της Move, ο οποίος αρχικοποιεί τις ιδιότητες της ανάλογα με τα ορίσματα που δέχεται. Επιπλέον, δημιουργήσαμε getters και setters για να υπάρχει πρόσβαση στις ιδιότητες της κάθε κίνησης (**row**, **col**, **value**).

Κλάση Player

Έπειτα, υλοποιήσαμε μία κλάση Player, η οποία περιγράφει τον παίκτη «Υπολογιστή». Αυτή διαθέτει ως ιδιότητες το μέγιστο βάθος αναζήτησης (**maxDepth**) και το χρώμα του παίκτη (**playercolour**). Σε αυτήν υλοποιείται και ο αλγόριθμος minimax με πριόνισμα α-β για την εύρεση της καλύτερης δυνατής κίνησης του υπολογιστή στο παιχνίδι. Αρχικά, ορίζουμε ότι ο παίκτης που θα έχει τα μαύρα πιόνια έχει σκοπό να μεγιστοποιήσει την ευρετική του συνάρτηση. Για να το πετύχει αυτό καλεί τη συνάρτηση max. Αντίθετα, ο παίκτης με τα άσπρα πιόνια επιδιώκει να ελαχιστοποιήσει την ευρετική συνάρτηση, καλώντας τη min αντίστοιχα. Η συνάρτηση max παίρνει ως ορίσματα το ταμπλό, το βάθος αναζήτησης και τα α, β για το πριόνισμα. Αυτή ελέγχει αν βρισκόμαστε σε τελική κατάσταση ή αν έχει φτάσει στο μέγιστο βάθος. Σε περίπτωση που τίποτα από αυτά δεν έχει συμβεί, δημιουργεί μια λίστα με όλα τα παιδιά του κόμβου, τα οποία αποτελούν τις επόμενες κινήσεις του max. Διατρέχοντας τη λίστα, για κάθε παιδί καλεί τη min του επόμενου επιπέδου, ώστε να βρει την ελάχιστη τιμή της ευρετικής για τις επόμενες κινήσεις του αντιπάλου. Διαλέγει την καλύτερη κίνηση, δηλαδή αυτή με τη μεγαλύτερη τιμή ευρετικής για τον max. Τέλος, ελέγχει ποιο από τα α, β είναι μεγαλύτερο για να γίνει το επιθυμητό πριόνισμα. Αντίστοιχα λειτουργεί και η συνάρτηση min. Σε αυτήν την περίπτωση, βρίσκει την καλύτερη κίνηση του αντιπάλου και επιστρέφει την κίνηση με τη μικρότερη ευρετική.

Κλάση Board

Η κλάση Board υλοποιεί το ταμπλό και τις καταστάσεις του κατά τη διάρκεια του παιχνιδιού. Θέτει **σταθερές**: τα μαύρα πιόνια ως **1**, τα λευκά ως **-1** και την κενή θέση ως **0**. Τοποθετεί τα 4 πρώτα πιόνια στο κέντρο του ταμπλό και θέτει τις υπόλοιπες θέσεις κενές. Διαθέτει έναν δισδιάστατο πίνακα (**gameboard**) 8 x 8, τον τελευταίο παίκτη

(lastPlayer) και την τελευταία κίνηση που πραγματοποιήθηκε (lastMove). Η ίδια, πέρα από τον constructor για την αρχικοποίηση του πίνακα, διαθέτει και τις ακόλουθες μεθόδους :

- **isValidMove(int row, int col, int colour)**

Η μέθοδος αυτή επιστρέφει μια αληθοτιμή True/False , η οποία ελέγχει το αν μια κίνηση του παιχνιδιού είναι έγκυρη ή όχι. Αρχικά, ελέγχει αν οι συντεταγμένες του κελιού στο οποίο θέλουμε να τοποθετήσουμε το πιόνι είναι εντός ορίων του πίνακα, δηλαδή του ταμπλό. Έπειτα, ελέγχει στις 8 διαφορετικές κατευθύνσεις αν υπάρχει τουλάχιστον μία απαραίτητη αναστροφή των πιονιών, ώστε να καταστεί έγκυρη η κίνηση.

- **getReversePoints(int row, int col, int colour)**

Η μέθοδος αυτή επιστρέφει μία λίστα, η οποία περιέχει τις συντεταγμένες των κελιών που χρειάζονται αναστροφή. Για να το πετύχει αυτό ελέγχει ξανά τις 8 κατευθύνσεις προσθέτοντας στη λίστα τα κατάλληλα κελιά. Αυτά είναι της μορφής Point που εκφράζει ένα ζευγάρι συντεταγμένων.

- **makeMove(int row, int col, int colour)**

Η μέθοδος αυτή δεν επιστρέφει κάτι αλλά πραγματοποιεί μια κίνηση του παιχνιδιού. Αρχικά, ελέγχει την εγκυρότητα πραγματοποίησης της κίνησης με τη βοήθεια της **isValidMove** και στη συνέχεια ενημερώνει τις ιδιότητες της κλάσης **Board** με βάση την κίνηση που έγινε. Τέλος, εκτελεί την αναστροφή των κατάλληλων πιονιών από το ένα χρώμα στο άλλο.

- **ArrayList<Board> getChildren(int colour)**

Η μέθοδος αυτή λαμβάνει ως όρισμα το χρώμα ενός παίκτη και επιστρέφει μια λίστα με τις επόμενες δυνατές κινήσεις του παίκτη, τα παιδιά δηλαδή του κόμβου. Για να το πετύχει αυτό διατρέχει όλα τα κελιά του πίνακα και ελέγχοντας την εγκυρότητα κάθε θέσης προσθέτει το κελί στη λίστα.

- **int evaluate()**

Η μέθοδος αυτή βρίσκει την ευρετική, αξιολογώντας την κάθε κίνηση. Ειδικότερα, μετράει πόσα μαύρα και πόσα άσπρα πιόνια υπάρχουν στο ταμπλό και επιστρέφει τη διαφορά τους. Με αυτόν τον τρόπο γνωρίζουμε ποιον παίκτη ευνοεί περισσότερο αυτή η κίνηση.

- **boolean isTerminal()**

Η μέθοδος αυτή ελέγχει αν το παιχνίδι βρίσκεται στην τερματική του κατάσταση και επιστρέφει μια αντίστοιχη αληθοτιμή True/False. Το παιχνίδι τελειώνει στην περίπτωση που όλα τα κελιά του πίνακα είναι γεμάτα, οπότε η

μέθοδος διατρέχει τον πίνακα για να ελέγξει αν υπάρχει έστω και ένα κενό κελί. Επίσης, μπορεί να τελειώσει και όταν κανένας παίκτης δε μπορεί να παίξει, δηλαδή δε διαθέτει έγκυρες κινήσεις. Για αυτό, η μέθοδος ελέγχει αν η λίστα που επιστρέφει η `getChildren` για τα άσπρα και τα μαύρα πιόνια είναι κενή.

- `void print()`
Η μέθοδος αυτή εκτυπώνει το ταμπλό και την κατάσταση του παιχνιδιού στο τερματικό.
- `getters και setters`
Υπάρχουν κατάλληλοι getters και setters για την τελευταία κίνηση `LastMove`, το ταμπλό `GameBoard`, και τον παίκτη που έπαιξε τελευταίος `lastPlayer`, ώστε να έχουμε πρόσβαση σε αυτές τις μεταβλητές.

Κλάση Main

Μέσω της κλάσης Main, εμφανίζονται αρχικά προτροπικά μηνύματα προς τον χρήστη, αναφορικά με το αν επιθυμεί να παίξει πρώτος καθώς και να επιλέξει το μέγιστο βάθος αναζήτησης του αλγορίθμου. Έπειτα, ανατείνονται πούλια κατάλληλου χρώματος στον κάθε παίκτη. Όσο το ταμπλό δεν βρίσκεται σε τελική κατάσταση, ελέγχεται αν υπάρχουν επιτρεπτές κινήσεις για τον κάθε παίκτη. Αν ένας από τους δύο παίκτες δεν έχει διαθέσιμη κίνηση, τότε η σειρά του παραχωρείται στον άλλο παίκτη. Αν κανένας από τους δύο δεν έχει επιτρεπτές κινήσεις, το παιχνίδι τερματίζει. Η Main λαμβάνει, από την GUI, την κίνηση του παίκτη – ανθρώπου και παραχωρεί τη σειρά στο AI, το οποίο αποφασίζει κατάλληλα, μέσω του αλγορίθμου Minimax, την κίνηση που θα κάνει. Η σειρά αλλάζει κατάλληλα συνεχώς, ώσπου στο τέλος, εμφανίζεται μήνυμα με το score του νικητή ή μήνυμα ισοπαλίας και το παιχνίδι τελειώνει.

Κλάση Point

Η κλάση Point χρησιμοποιείται μέσα στην κλάση `Board` και αποθηκεύει τις συντεταγμένες ενός κελιού και ειδικότερα τις συντεταγμένες των θέσεων του ταμπλό που χρειάζονται αναστροφή.

Κλάση GUI

Στην κλάση GUI χρησιμοποιήθηκε ελεύθερο λογισμικό από το διαδίκτυο. Η κλάση αυτή υλοποιεί τη διεπαφή του παιχνιδιού όπως για παράδειγμα το παράθυρο, το ταμπλό και

τα πιόνια και η λειτουργία της είναι μόνο να λαμβάνει input από τον παίκτη – άνθρωπο και να μεταβιβάζει το input αυτό στην κλάση Main, όπου και γίνεται η επεξεργασία της κάθε κίνησης.

4. Μέθοδοι Τεχνητής Νοημοσύνης

Ο αλγόριθμος Τεχνητής Νοημοσύνης που χρησιμοποιήθηκε για την εύρεση της καλύτερης κίνησης είναι ο **Minimax με πριόνισμα α-β**.

Ο παίκτης με τα μαύρα πιόνια είναι ο **Max**, δηλαδή επιδιώκει να μεγιστοποιήσει το όφελός του, ενώ ο παίκτης με τα άσπρα πιόνια είναι ο **Min**, δηλαδή επιδιώκει να ελαχιστοποιήσει το όφελός του.

Πρώτα, αρχικοποιείται το **βάθος αναζήτησης** του αλγορίθμου με 0, το **άλφα** με πλην άπειρο και το **βήτα**, για το πριόνισμα, με συν άπειρο. Αφού πάντα ο πρώτος παίκτης είναι αυτός με τα μαύρα πιόνια, καλείται αρχικά η συνάρτηση **max** η οποία αναζητά την κίνηση με την μέγιστη ευρετική αξία.

Αν το παιχνίδι βρίσκεται σε τελική κατάσταση ή έχουμε φτάσει σε μέγιστο βάθος αναζήτησης, τότε υπολογίζεται η τιμή της ευρετικής και επιστρέφεται η νέα κίνηση. Έπειτα, δημιουργούνται τα παιδιά της κίνησης του παίκτη **Max**, δηλαδή όλες οι επόμενες δυνατές κινήσεις. Αν δεν έχει παιδιά, τότε καλείται απευθείας η **min** του αντιπάλου στο επόμενο βάθος. Αλλιώς, διατρέχουμε τη λίστα των παιδιών και για κάθε ένα από αυτά, καλείται η αντίστοιχη συνάρτηση **min** του αντιπάλου.

Ο **Max** επιλέγει την κίνηση με την μεγαλύτερη αξία που επιστράφηκε από την κλήση της **min**. Σε περίπτωση ισοπαλίας των τιμών της ευρετικής, επιλέγεται τυχαία μία κίνηση. Ο **Max** ενημερώνει την μεταβλητή **άλφα** με την μεγαλύτερη αξία που βρέθηκε προηγουμένως και τη συγκρίνει με την μεταβλητή **βήτα**. Σε περίπτωση που η τιμή της **βήτα** είναι μικρότερη ή ίση από αυτήν της **άλφα** σταματάμε την αναζήτηση στον συγκεκριμένο κλάδο του δέντρου, καθώς γνωρίζουμε ότι σε αυτόν τον κλάδο υπάρχει κίνηση που είναι καλύτερη για τον αντίπαλο, οπότε δεν έχει νόημα να επιλέξουμε κίνηση από εκεί.

Στη συνέχεια, καλείται η συνάρτηση **min**, η οποία λειτουργεί με αντίστοιχο τρόπο, επιδιώκοντας να ελαχιστοποιήσει την τιμή της ευρετικής. Στο πριόνισμα ενημερώνει το **βήτα** με τη μέγιστη αξία που βρέθηκε, το συγκρίνει με το **άλφα**, και αν η τιμή του είναι μεγαλύτερη ή ίση από αυτήν της **άλφα** σταματάει την αναζήτηση στον συγκεκριμένο κλάδο.

Οι συναρτήσεις καλούνται αναδρομικά, αναλόγως με την εναλλαγή της σειράς των παικτών.

5. Παραδείγματα Χρήσης

Παρακάτω παρατίθενται στιγμιότυπα μιας παρτίδας με παίκτη-Άνθρωπο στα μαύρα πούλια και τον Υπολογιστή στα άσπρα καθώς και `maximumdepth = 4`.

